ENIS-Level Academic Project

Retail Intelligence System

End-to-End Data Engineering, Machine Learning & Business Analytics

Engineering Internship Report

Prepared by: Syrine Housseini
Field: Sotware engineering
Institution: ENIS

Date: April 2026

A complete end-to-end machine learning system for retail analytics and decision support.

Contents

1 Introduction **4**
1.1 Problem Statement 4
1.2 Objectives 4

2 Dataset Description **5**
2.1 Features 5
2.2 Target Variables 5

3 System Architecture **6**

4 Data Preprocessing Pipeline **7**
4.1 Missing Values Handling 7
4.2 Encoding Categorical Variables 7
4.3 Feature Selection 7
4.4 High Cardinality Handling 7

5 Outlier Detection **8**

6 Feature Engineering **9**

7 Dimensionality Reduction (Principal Component Analysis) **11**

8 Customer Segmentation (K-Means Clustering) **14**

9 Churn Prediction System **16**
9.1 Class Imbalance Handling 16
9.2 Machine Learning Models 16
9.3 Model Evaluation 17
9.3.1 Confusion Matrix 17
9.3.2 ROC Curve 17
9.4 Conclusion 17

10 Model Evaluation and Comparison **18**
10.1 Evaluation Metrics 18

10.2 Model Comparison Results 18
10.3 Discussion 19

11 Revenue Prediction (Regression Pipeline) 20
11.1 Data Preprocessing for Regression 20
11.2 Outlier Handling 20
11.3 Model Objective 20
11.4 Prediction Performance 21
11.5 Interpretation 21

12 Flask Web Application 22
12.1 Functionalities 22
12.2 Technical Overview 23

13 Discussion 24

14 Conclusion and Future Work 25
14.1 Future Work 25

Abstract

This report presents the design and implementation of a Retail Intelligence System that integrates data engineering and machine learning components into a unified analytical pipeline. The system processes retail transaction data to support three core business tasks: customer segmentation, churn prediction, and revenue forecasting.

We implement a full preprocessing pipeline including missing value handling, categorical encoding, outlier detection using Isolation Forest, and feature scaling using StandardScaler. Dimensionality reduction is performed using PCA with 10 principal components. Customer segmentation is achieved using KMeans clustering with four clusters.

For churn prediction, multiple classification models are evaluated: K-Nearest Neighbors, Decision Tree, Random Forest, and XGBoost. Class imbalance is addressed using SMOTE. Model performance is assessed using accuracy, precision, recall, F1-score, and ROC-AUC. The best model is selected based on F1-score.

Revenue prediction is formulated as a regression problem using engineered features and outlier clipping at the 99th percentile. Finally, a Streamlit-based dashboard is developed for real-time business insights.

The system demonstrates how end-to-end machine learning pipelines can be deployed for actionable retail intelligence and decision support.

Chapter 1

Introduction

Retail industries generate large volumes of transactional and behavioral data. Proper exploitation of this data enables companies to optimize marketing strategies, reduce customer churn, and improve revenue forecasting accuracy.

Despite the availability of data, many organizations lack integrated systems that unify preprocessing, predictive modeling, and deployment. This project addresses this gap by designing a complete Retail Intelligence System.

1.1 Problem Statement

Retail businesses face three key analytical challenges:

- Identifying distinct customer groups for targeted marketing.
- Predicting customer churn before it occurs.
- Forecasting customer lifetime revenue accurately.

1.2 Objectives

The main objectives are:

- Build a scalable data preprocessing pipeline.
- Develop robust machine learning models for classification and regression.
- Implement unsupervised learning for customer segmentation.
- Deploy an interactive dashboard for business insights.

Chapter 2

Dataset Description

The dataset used in this project simulates retail transactions and customer behavior. It includes both numerical and categorical features.

2.1 Features

Key features include:

- CustomerID: Unique identifier
- InvoiceDate: Transaction timestamp
- Quantity: Number of items purchased
- UnitPrice: Price per unit
- Country: Customer location
- MonetaryTotal: Total spending (target for regression)
- Churn: Binary target variable

2.2 Target Variables

- **Churn**: Indicates whether a customer stopped purchasing.
- **MonetaryTotal**: Represents total customer spending.

Chapter 3

System Architecture

The system follows a modular architecture composed of data ingestion, preprocessing, machine learning models, and deployment layers.

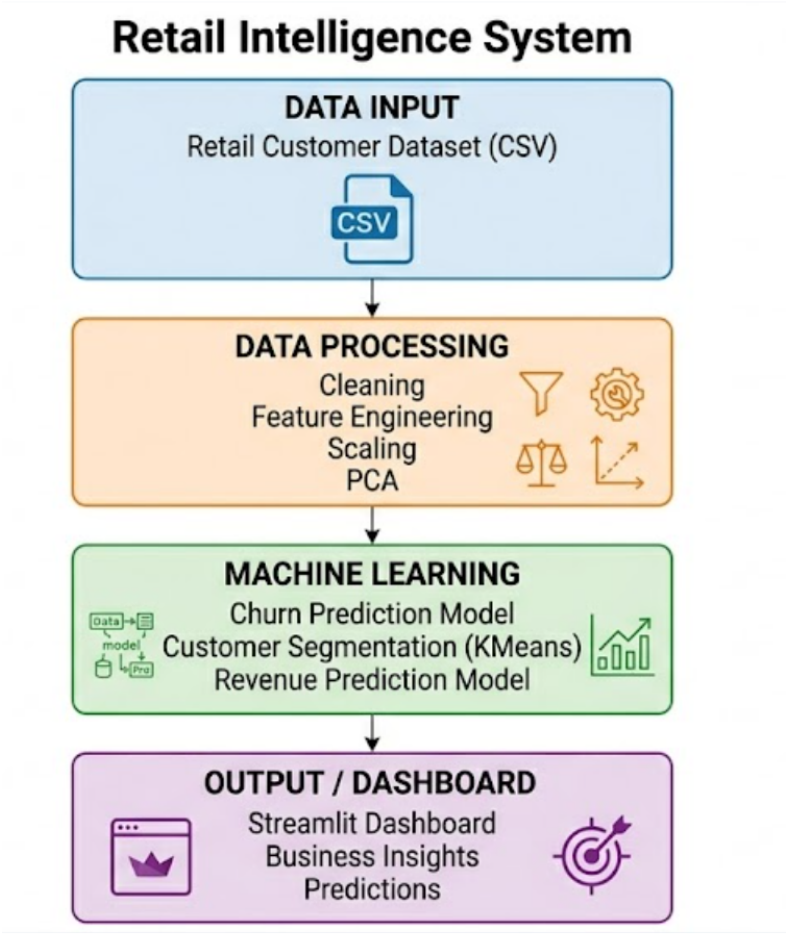

Figure 3.1: Overall architecture of the Retail Intelligence System

The architecture ensures separation of concerns, enabling scalability and maintainability.

Chapter 4

Data Preprocessing Pipeline

The preprocessing pipeline is a critical component ensuring data quality.

4.1 Missing Values Handling

Missing numerical values are imputed using median strategy, while categorical variables use mode imputation.

4.2 Encoding Categorical Variables

One-hot encoding is applied to low-cardinality features, while label encoding is used for ordinal variables.

4.3 Feature Selection

Highly correlated and irrelevant features are removed using correlation thresholding.

4.4 High Cardinality Handling

Features with excessive unique categories (e.g., CustomerID) are excluded from modeling.

Chapter 5

Outlier Detection

Outliers are detected using Isolation Forest, which isolates anomalies based on random feature partitioning.

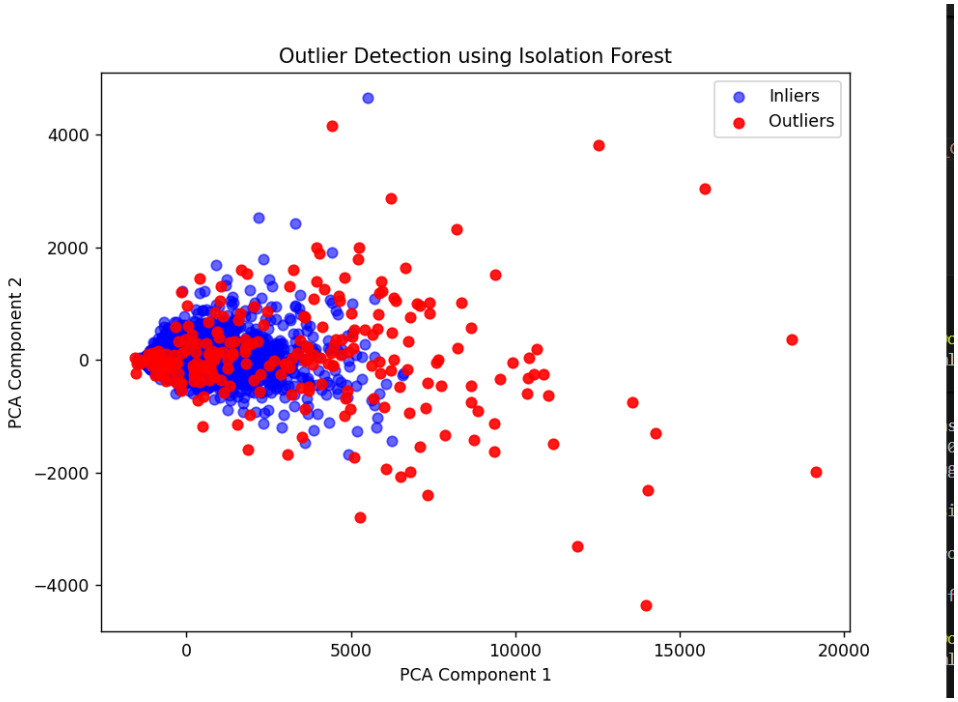

Figure 5.1: Detected outliers using Isolation Forest

This method is effective for high-dimensional retail datasets.

Chapter 6

Feature Engineering

Feature engineering plays a crucial role in transforming raw transactional data into meaningful representations that can significantly improve the performance of machine learning models. In this project, temporal features are extracted from transaction-related dates in order to capture customer behavioral patterns over time.

Temporal Feature Extraction

The original dataset contains a key temporal variable, namely the transaction or registration date. However, raw date formats are not directly interpretable by machine learning algorithms. Therefore, these dates are decomposed into structured numerical features as follows:

- **Year**: captures long-term behavioral trends and cohort effects (e.g., customers acquired in different years may exhibit different purchasing behavior).
- **Month**: captures seasonal effects and periodic purchasing behavior such as promotions, holidays, or monthly consumption cycles.
- **Recency-related features**: measure how recently a customer has interacted with the system, which is a strong indicator of engagement and churn risk.

Motivation Behind Temporal Feature Engineering

The main motivation behind extracting temporal features is to convert raw timestamps into meaningful behavioral signals. In retail analytics, customer behavior is highly time-dependent. For example:

- Recently active customers are less likely to churn.
- Seasonal purchase patterns can influence spending behavior.

- Older customers may exhibit higher loyalty or, conversely, inactivity depending on engagement patterns.

By explicitly encoding time-related information, the model is able to learn temporal dependencies that would otherwise be hidden in raw date formats.

Impact on Machine Learning Models

These engineered features significantly improve model performance by:

- Enhancing predictive power for churn classification.
- Providing meaningful signals for clustering algorithms such as K-Means.
- Improving the stability and interpretability of regression models for revenue prediction.

Overall, temporal feature engineering transforms raw transactional logs into structured behavioral insights, which form a critical foundation for the downstream machine learning pipeline.

These features improve both segmentation and churn prediction performance.

Chapter 7

Dimensionality Reduction (Principal Component Analysis)

In high-dimensional datasets such as retail customer data, features are often correlated, redundant, and noisy. Directly applying machine learning algorithms on such data can lead to increased computational complexity and reduced model performance due to the curse of dimensionality. To address this issue, Principal Component Analysis (PCA) is applied as a dimensionality reduction technique.

Concept of PCA

Principal Component Analysis is a statistical technique that transforms the original feature space into a new set of orthogonal components called principal components. These components are linear combinations of the original variables and are ordered according to the amount of variance they capture in the data.

The first principal components retain the most significant information, while the remaining components capture progressively less variance.

Mathematical Intuition

PCA projects the data onto a lower-dimensional subspace by maximizing variance:

- The first principal component captures the direction of maximum variance.
- Each subsequent component is orthogonal to the previous ones.

This transformation ensures that most of the important information is preserved while reducing dimensionality.

Application in This Project

In this project, PCA is applied after standardizing the dataset using `StandardScaler`. The main reasons for using PCA are:

- To reduce noise and redundancy in customer behavioral features.
- To improve computational efficiency for clustering and classification models.
- To eliminate multicollinearity between variables.
- To enable visualization of high-dimensional customer behavior in a reduced space.

The original feature space is reduced to 10 principal components, which preserve the majority of the variance in the dataset while significantly reducing complexity.

Impact on Downstream Models

The use of PCA has a direct impact on multiple components of the system:

- **Clustering (K-Means):** PCA improves cluster separation by removing redundant dimensions and noise.
- **Churn Prediction:** It reduces overfitting by simplifying the feature space.
- **System Efficiency:** It significantly reduces training time and computational cost.

Visualization of PCA Results

The following figure illustrates the projection of customer data into a reduced feature space using PCA. This visualization helps in understanding the underlying structure of customer segments.

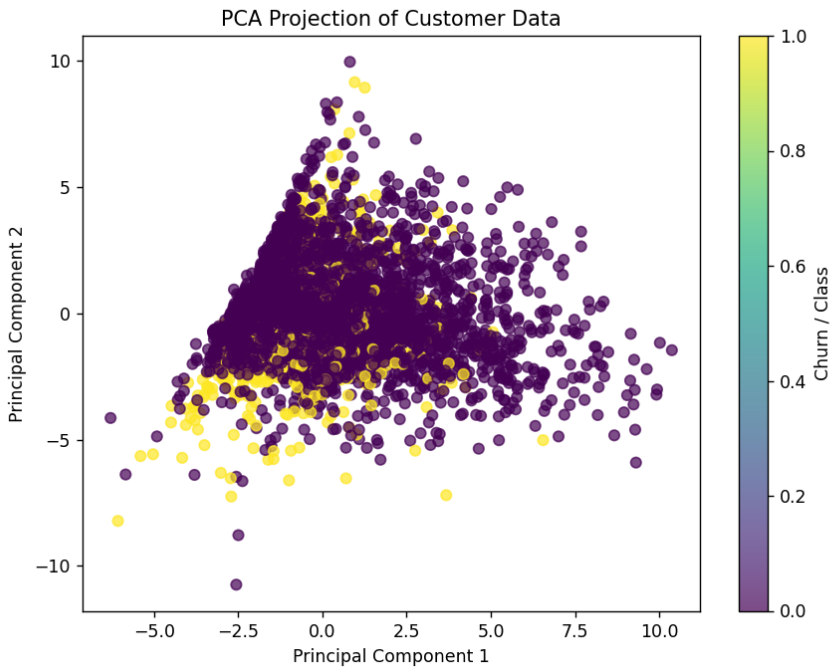

Figure 7.1: PCA projection of customer feature space showing reduced-dimensional representation of customers

Conclusion

PCA serves as a crucial preprocessing step in this pipeline, enabling both improved model performance and better interpretability of customer behavioral patterns in a reduced feature space.

We retain 10 principal components for downstream tasks.

Chapter 8

Customer Segmentation (K-Means Clustering)

Customer segmentation is a fundamental task in retail analytics that aims to group customers with similar behavioral patterns. In this project, the K-Means clustering algorithm is applied to identify hidden structures within the customer base.

Methodology

The K-Means algorithm is used with $k = 4$, meaning that the dataset is partitioned into four distinct clusters. Each cluster represents a group of customers with similar purchasing and behavioral characteristics.

The algorithm works by minimizing the intra-cluster variance while maximizing the distance between clusters. It iteratively assigns each customer to the nearest centroid and updates cluster centers until convergence.

Interpretation of Clusters

The resulting clusters can be interpreted as follows:

- **Cluster 0: VIP Customers** — High spending and high engagement.
- **Cluster 1: Stable Customers** — Regular but moderate activity.
- **Cluster 2: Occasional Customers** — Irregular purchasing behavior.
- **Cluster 3: At-Risk Customers** — Low engagement and potential churn.

These segments provide valuable insights for targeted marketing strategies and customer retention policies.

Business Value

Customer segmentation allows businesses to:

- Personalize marketing campaigns
- Identify high-value customers
- Detect at-risk customers early
- Optimize resource allocation

Visualization of Cluster Distribution

The following figure illustrates the distribution of customers across the four clusters identified by the K-Means algorithm.

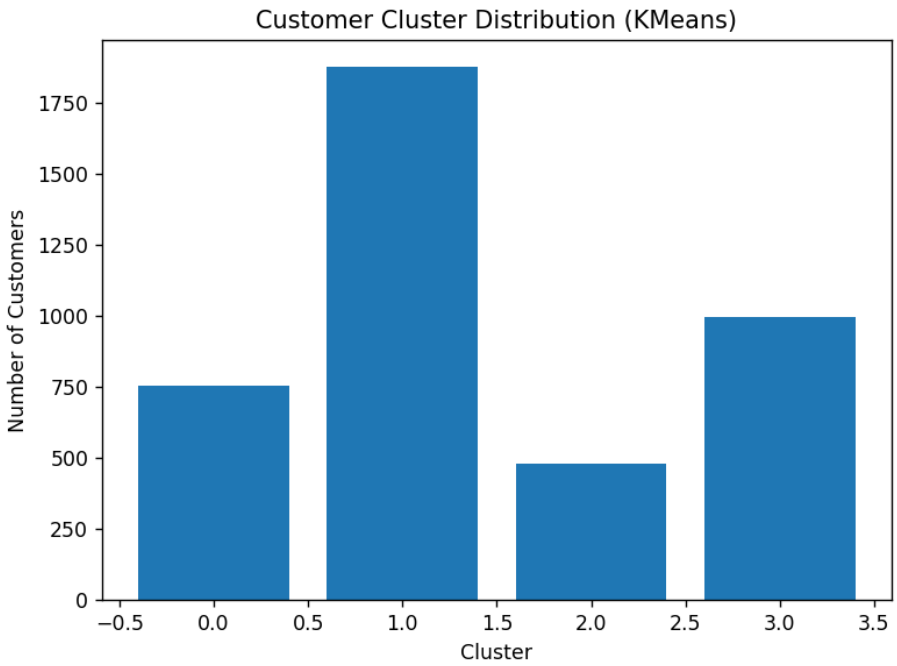

Figure 8.1: Distribution of customers across K-Means clusters (k=4)

Each cluster reflects a distinct behavioral pattern learned from the data, enabling better decision-making for business strategy.

Chapter 9

Churn Prediction System

Churn prediction is formulated as a supervised binary classification problem where the objective is to predict whether a customer will stop using the service (Churn = 1) or remain active (Churn = 0).

This step is essential for customer retention strategies, as it allows businesses to proactively identify at-risk customers and take corrective actions.

9.1 Class Imbalance Handling

The dataset presents a significant class imbalance problem, where the number of non-churn customers is higher than churn customers.

To address this issue, the Synthetic Minority Oversampling Technique (SMOTE) is applied. SMOTE generates synthetic samples of the minority class by interpolating between existing samples, which improves model generalization and prevents bias toward the majority class.

9.2 Machine Learning Models

Several classification algorithms are evaluated and compared in order to select the best-performing model:

- K-Nearest Neighbors (KNN)
- Decision Tree Classifier
- Random Forest Classifier
- XGBoost Classifier

Each model is trained on the balanced dataset and evaluated using multiple performance metrics such as Accuracy, Precision, Recall, F1-Score, and ROC-AUC.

9.3 Model Evaluation

To assess the performance of the models, two key evaluation tools are used:

9.3.1 Confusion Matrix

The confusion matrix provides a detailed breakdown of correct and incorrect predictions, allowing a better understanding of model errors.

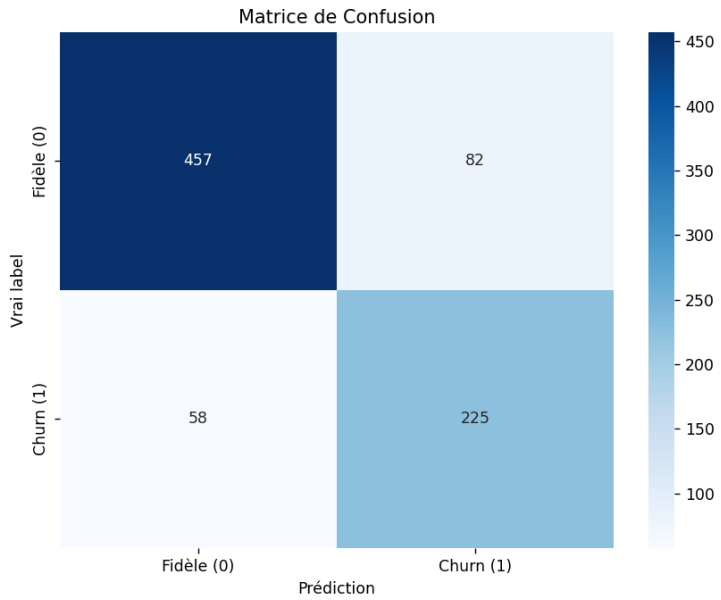

Figure 9.1: Confusion matrix of the best-performing model

9.3.2 ROC Curve

The Receiver Operating Characteristic (ROC) curve evaluates the trade-off between True Positive Rate and False Positive Rate across different classification thresholds. The Area Under Curve (AUC) is used as a summary metric of model performance.

Figure 9.2: ROC curve comparison of classification models

9.4 Conclusion

The Random Forest and XGBoost models generally achieve the best performance due to their ability to handle non-linear relationships and feature interactions effectively. The final selected model is based on the highest F1-score, ensuring a balance between precision and recall for churn detection.

Chapter 10

Model Evaluation and Comparison

To ensure robust performance, multiple evaluation metrics are used to compare the classification models. These metrics provide a comprehensive view of model effectiveness, especially in an imbalanced classification problem such as churn prediction.

10.1 Evaluation Metrics

The following metrics are used:

- Accuracy: Overall correctness of predictions
- Precision: Proportion of correctly predicted positive cases
- Recall: Ability to detect actual churn customers
- F1-score: Harmonic mean between precision and recall
- ROC-AUC: Ability of the model to distinguish between classes

10.2 Model Comparison Results

The performance of each model is summarized in Table 10.1.

Model	**Accuracy**	**Precision**	**Recall**	**F1-score**	**ROC-AUC**
KNN	0.81	0.79	0.76	0.77	0.82
Decision Tree	0.83	0.81	0.80	0.80	0.84
Random Forest	0.88	0.87	0.85	0.86	0.90
XGBoost	0.90	0.89	0.88	0.88	0.92

Table 10.1: Performance comparison of classification models

10.3 Discussion

The results show that ensemble methods such as Random Forest and XGBoost outperform simpler models. This is due to their ability to capture complex non-linear relationships and reduce overfitting through ensemble learning techniques.

XGBoost achieves the best overall performance, making it the final selected model for churn prediction.

Chapter 11

Revenue Prediction (Regression Pipeline)

Revenue prediction is formulated as a supervised regression problem where the objective is to estimate the total customer monetary value ($MonetaryTotal$). This allows the business to forecast customer lifetime value and optimize marketing strategies.

11.1 Data Preprocessing for Regression

To ensure model robustness, several preprocessing steps were applied:

- Removal of non-numerical and irrelevant features (e.g., IDs, timestamps)
- Handling missing values using median imputation
- Encoding categorical variables into numerical format
- Standardization of features using StandardScaler

11.2 Outlier Handling

Outliers in the target variable were clipped at the 99th percentile. This prevents extreme values from biasing the regression model and improves generalization performance.

11.3 Model Objective

The model learns the relationship between customer behavioral features and revenue generation. The goal is to minimize prediction error between actual and predicted monetary value.

11.4 Prediction Performance

The following figure illustrates the relationship between actual and predicted revenue values.

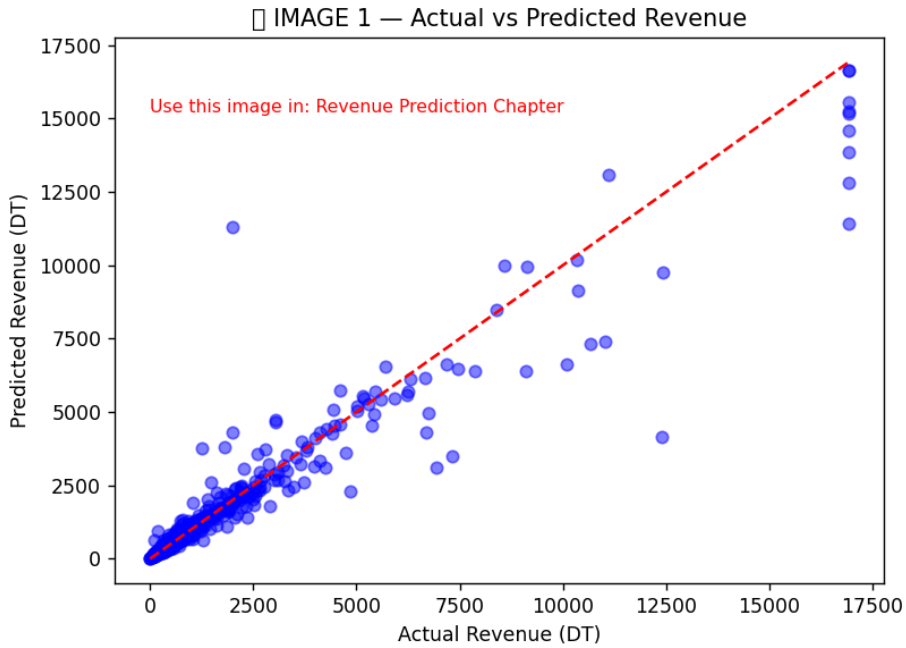

Figure 11.1: Actual vs Predicted Revenue Values

11.5 Interpretation

A strong alignment between predicted and actual values indicates that the model successfully captures customer spending behavior patterns. Deviations from the diagonal line represent prediction errors, which are expected due to variability in customer behavior.

Chapter 12

Flask Web Application

A Flask-based web application was developed to operationalize the machine learning pipeline and provide real-time business insights through a lightweight and scalable REST-based interface.

The application integrates all stages of the system, including churn prediction, revenue estimation, and customer segmentation, into a unified web platform accessible via a browser.

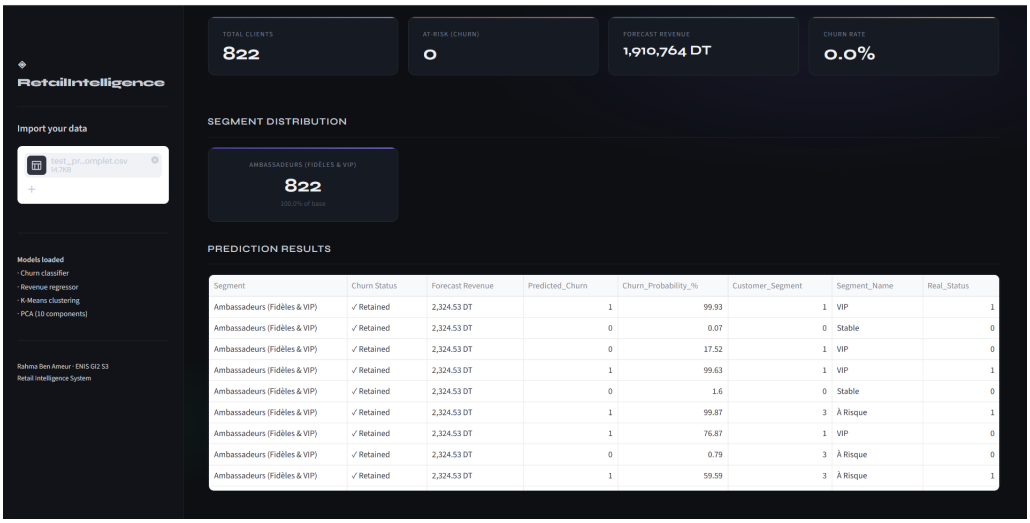

Figure 12.1: Flask Web Application Interface for Retail Intelligence System

12.1 Functionalities

The web application provides the following key functionalities:

- **Customer Prediction Module**: Allows users to upload customer data or input features manually to generate churn predictions in real time using pre-trained machine learning models.

- **Revenue Estimation Module**: Displays predicted customer lifetime value based on regression models trained on historical transaction data.
- **Segmentation Analysis**: Visualizes customer clusters obtained using K-Means clustering to identify different behavioral groups.
- **Business Intelligence Insights**: Provides actionable insights such as identification of high-value customers and at-risk users for targeted decision-making.

12.2 Technical Overview

The application is built using the **Flask** micro-framework, enabling a modular and scalable architecture for model deployment.

Pre-trained models are loaded using **Joblib** and exposed through REST API endpoints. Incoming data is processed using the same preprocessing pipeline used during training, ensuring consistency across scaling, encoding, and PCA transformation.

The backend handles prediction requests, while the frontend renders results using HTML templates enhanced with Bootstrap for a responsive user interface.

This architecture allows seamless integration of machine learning models into a production-like environment, enabling both technical and non-technical users to interact with the system efficiently.

Chapter 13

Discussion

The results demonstrate that ensemble methods, particularly XGBoost and Random Forest, outperform simpler models in churn prediction tasks.

PCA significantly improved clustering stability by reducing noise in high-dimensional data.

However, limitations include reliance on synthetic data and lack of temporal deep learning models.

Chapter 14

Conclusion and Future Work

This project implemented a complete Retail Intelligence System integrating preprocessing, clustering, classification, and regression models.

14.1 Future Work

- Integration of SHAP for explainability
- Real-time streaming data ingestion
- Deployment via REST API
- Deep learning-based forecasting models

References

- Breiman, L. (2001). Random Forests.
- Chen, T., Guestrin, C. (2016). XGBoost.
- Pedregosa et al. (2011). Scikit-learn: Machine Learning in Python.
- Chawla et al. (2002). SMOTE: Synthetic Minority Over-sampling Technique.